Fintech Engineer Take-Home Assessment

Apache Fineract (Java/Spring Boot) + Python ERP and Integration: Reliable Financial Integration

Time budget	About 30 hours of effort. Submit within 7 calendar days of receiving this.
Submission	Git repo + Fineract patch + docs + reproducible evidence
We are testing	Java/Spring Boot skill inside Fineract, Python engineering skill, and financial correctness under failure

1. Objective

You are joining a team that connects an **Apache Fineract core banking system** to an **ERP** and to **third-party financing partners**. In this scenario the **ERP owns the accounting master data** (the chart of accounts). Fineract needs matching GL accounts to post loan transactions.

You will (1) **extend Fineract in Java/Spring Boot** so ERP-owned GL accounts can be safely synchronized into it, (2) **build the Mock ERP and the Integration Service in Python**, and (3) prove the whole system stays correct under duplicates, timeouts, partial success and concurrency.

This is **not** a production banking platform. **We evaluate correctness of the Core and Reliability tiers over breadth or polish.** So scope carefully.

2. Tiers

Finish each tier before starting the next, and state in your README which you completed.

Tier	Parts	Suggested time
1. Core (required)	A-G	~45%

Tier	Parts	Suggested time
2. Reliability (required)	H-K	~45%
3. Stretch (optional)	L	~10%

If you run out of time, document what you would do next and why.

3. Technology and language requirements

Required: Apache Fineract, **Java/Spring Boot**, **Python**, PostgreSQL or MySQL, REST, Git, Docker Compose (one command starts everything).

Language ownership (both are mandatory and neither can substitute for the other):

Component	Language	Notes
Fineract customization	**Java / Spring Boot**	A change inside Fineract itself (Part A)
Mock ERP	**Python**	Any framework or none (FastAPI, Django REST, Flask, aiohttp…); justify your choice
Integration Service	**Python**	Same rule
Tests for Python components	**Python**	`pytest` or equivalent

Calling existing Fineract APIs from Python, or wrapping them in a thin Java endpoint, does **not** satisfy the Java requirement. A Python wrapper around Fineract does **not** satisfy the Python requirement.

Optional: Redis, a worker (Celery or similar), Kafka/RabbitMQ, OpenTelemetry, Prometheus/Grafana. A queue or worker is **not required**. Justify every major component; explaining why you did *not* need one earns credit.

Environment rules

- Use an official Apache Fineract release tag. **Pin the exact tag, JDK version and build tool**, and state them in the README. Builds are version-sensitive.
- You build Fineract from source with your patch applied and bake it into a custom Docker image. Pulling an official image is fine for the unmodified baseline only.

- The Fineract database starts **empty** (no pre-populated clients, loans or GL accounts). Setup must be scripted and repeatable. Internet access is permitted.

4. Business scenario

A fintech offers SME asset financing.

- **ERP (accounting master):** owns the chart of accounts and records journal entries. You build a **Mock ERP**.
- **Fineract:** system of record for clients, loans, disbursements, repayments, balances. Its GL accounts must mirror the ERP's.
- **Financing partner:** submits requests to your **Integration Service** over REST.

Architecture is your choice. Typical flow: ERP chart of accounts → Integration Service → Fineract GL (via your Java customization); Partner → Integration Service → (Fineract, ERP) for loan operations; reconciliation reads both.

---

5. Tier 1: Core

Part A: ERP-owned GL accounts and the Fineract Java customization

Scenario. When an account is created in the ERP, the corresponding GL account must exist in Fineract before it is used in loan accounting.

```
ERP  POST /api/accounts
{ "code": "1200", "name": "Loan Receivable", "type": "ASSET",
  "currency": "RWF", "external_reference": "GL-1200" }
```

Java requirement. Modify or extend Fineract (Spring Boot) so that GL account synchronization has the following **behaviors, enforced inside Fineract**:

1. The ERP `external_reference` is persisted on the Fineract GL account and is **unique**. First check what your pinned Fineract version already supports; if it already stores a suitable reference, your customization must still provide the remaining behaviors below.

2. Creating a GL account with a reference that already exists and an **identical payload** returns the existing account (no duplicate).
3. The same reference with a **different payload** is rejected with a conflict (409 or Fineract's equivalent).
4. The endpoint is protected by Fineract's own permission model.
5. Any schema change ships as a **Liquibase** migration.
6. Behavior is covered by **Java tests** (at least one unit test and one integration test, run with JUnit).

You choose the mechanism (new endpoint, extension of the existing GL command handler, or a Fineract-side component). Document the choice and why.

Python side of the sync.

- Trigger is your choice (ERP webhook, polling, or explicit call); document it.
- Map ERP account types to Fineract GL types explicitly, in one tested place.
- Sync is **create-only**. Updates and deletions are out of scope.
- Sync status per account: `SYNC_PENDING -> SYNCED` (or `FAILED`), with retry if Fineract is down. Sending the same account 4 times must yield one ERP account, one Fineract GL account, one mapping.
- **Order matters:** GL accounts must be synced before the loan product (Part B) references them.

Evidence of Java work: a patch/diff against the upstream tag, the build log showing the custom image compiles and Java tests pass, a clean `docker compose up` starting the customized Fineract, and a scripted demo of behaviors 1 to 4. Dead code that is not exercised does not count.

Part B: Fineract environment

Configure at minimum:

- Office `Kigali Main Office`, currency `RWF`
- Loan product "SME Asset Finance" (suggested 18% annual, monthly repayments, 12 months), **single-disbursement** (no multi-tranche), using the **synced** GL accounts

- Accounting rules (cash-based or accrual: state which and why)
- An API user with least privilege

No fees, penalties or taxes are required (stretch item). Document every configuration decision.

Part C: Identities

Two business identifiers, neither a Fineract internal ID:

Identifier	Meaning	Stored in Fineract as
`client_uid` (e.g. `SME-000001`)	Client business ID	client `externalId`
`loan_reference` (e.g. `LN-2026-000001`)	Integration-level loan ID issued by your service	loan `externalId`

Keep a local mapping table. **Callers never see or send Fineract's numeric IDs**, and your API must not leak them as primary identifiers.

Demonstrate: create client → find by `client_uid` → validate (exists, active, correct name).

Part D: Loan lifecycle

Run one loan end to end: **application → approval → disbursement → repayments → final payoff → Closed (obligations met)**.

Suggested: 5,000,000 RWF, 18%, 12 months, at least three partial repayments, then a final payment clearing the balance. Capture Fineract's **actual** status and balances after every stage.

Part E: Integration API

Minimum (you may redesign and justify):

```
POST /api/accounts/sync                          (or equivalent trigger for Part A)
POST /api/clients
GET  /api/clients/{client_uid}
POST /api/loans                                  (application; approval here or separat
e)
GET  /api/loans/{loan_reference}
POST /api/loans/{loan_reference}/disburse
```

```
POST /api/loans/{loan_reference}/repayments
GET  /api/transactions/{external_reference}
POST /api/reconciliation
```

Required: authentication, validation, consistent error format, correlation ID on every request/response, OpenAPI docs (or equivalent).

Part F: External reference and idempotency

Every state-changing request (GL sync, client creation, loan application, approval, disbursement, repayment) carries an `external_reference`, unique per business operation and independent of Fineract IDs.

Case	Required behaviour
New reference	Process once
Same reference, same payload, finished	Return the original result (200, same body)
Same reference, same payload, in progress	202 with current status; never start a second execution
Same reference, different payload	409 Conflict

Uniqueness is scoped by operation type:

- **GL account:** unique per `external_reference`, and `code` is unique.
- **Client creation:** unique per `external_reference`. The same reference never creates two clients. A different reference may create another client only if its `client_uid` is also different; `client_uid` is unique at the database level, so reuse is rejected with 409.
- **Loan application:** unique per `external_reference`.
- **Disbursement is unique per loan.** A loan may be disbursed only once; concurrent requests with *different* references for the same loan still result in exactly one Fineract disbursement.
- **Repayment is unique per `external_reference`.** Many legitimate repayments may hit one loan.

Explain precisely what mechanism prevents double processing.

Part G: Mock ERP (Python)

```
POST /api/accounts                          (chart of accounts; idempotent on external_ref
erence)
GET  /api/accounts
GET  /api/accounts/{code}/balance
POST /api/journal-entries                   (idempotent on external_reference)
GET  /api/journal-entries/{external_reference}
```

Seed a chart of accounts (e.g. 1100 Bank, 1200 Loan Receivable, 4000 Interest Income). Entries must balance (debits = credits) or be rejected. The ERP enforces its own uniqueness; do not rely only on the caller.

---

6. Tier 2: Reliability

Part H: Accounting mapping

Derive journal entries **from Fineract's transaction data**, not your own interest calculations.

- **Disbursement:** DR Loan Receivable / CR Bank
- **Repayment:** DR Bank / CR Loan Receivable (principal) / CR Interest Income (interest)

Explain where the principal/interest split comes from (Fineract's transaction breakdown) and how overpayments and reversals would be handled. **Documenting this is required; demonstrating it is optional unless selected as a stretch item.** Mapping must be a pure, unit-tested function.

Part I: Duplicates and concurrency (mandatory)

I1. Sequential duplicates. Send the same disbursement 4 times. Show:

```
Requests received:        4
Fineract disbursements:   1
ERP journal entries:      1
Integration transactions: 1
```

I2. Concurrent disbursements. Fire at least 10 simultaneous disbursements for the same loan, (a) with the same reference and (b) with different references. Expected: exactly **one** disbursement. Explain where the final financial invariant is enforced. Distinguish the layers you rely on (API idempotency, integration-DB constraint, locking, Fineract `externalId`, Fineract loan state), say why one layer alone is not enough, and name the last line of defence. Evidence must come from an **automated parallel test**.

I3. Concurrent repayments. Fire simultaneous repayments with *different* references and confirm each applies exactly once. The final Fineract outstanding balance must equal the initial outstanding minus the repayment amounts **as recorded by Fineract** (its transaction breakdown), not merely the amounts your test sent. Duplicates of one reference must apply once.

Part J: Failure handling and recovery (mandatory)

Build a way to inject failures on demand. For each scenario show state **before, during and after recovery**.

#	Scenario	Questions
1	ERP down after Fineract success	Status? Retried? How does the ERP catch up? How is double posting prevented?
2	Fineract down	What does the caller get? Is anything persisted? Safe to retry?
3	Fineract timeout after sending	How do you know whether it succeeded? (Hint: send your reference as Fineract's `externalId` and look it up; do not blindly retry.)
4	ERP timeout after the entry was written	What happens on retry? Prove one journal entry exists.
5	Crash between Fineract success and the ERP call	How is it recovered on restart?

Model the lifecycle as an explicit **state machine** (e.g. `RECEIVED -> FINERACT_PENDING -> FINERACT_DONE -> ERP_PENDING -> COMPLETED`, plus `FAILED`, `NEEDS_RECONCILIATION`). Document allowed transitions, retry policy (backoff, max attempts) and what is safe to retry.

Recovery without a worker is acceptable. Persisted state plus an explicit recovery mechanism (startup sweep, scheduled job, admin endpoint) is enough, provided an interrupted transaction can be detected after restart and then completed or routed to reconciliation.

Part K: Reconciliation

`POST /api/reconciliation` compares Fineract and the ERP over a date range or set of references.

Status	Meaning
`MATCHED`	Present in both, same amount and type
`MISSING_IN_ERP`	In Fineract, not in ERP
`MISSING_IN_FINERACT`	In ERP, not in Fineract
`AMOUNT_MISMATCH`	Both present, amounts differ

Keying: compare per Fineract transaction (disbursement or repayment), matched to the ERP journal entry carrying the same `external_reference`. State how reversals are handled. Seed all four outcomes deliberately and show the report. Reconciliation is **read-only by default**; any repair action must be explicit and idempotent. This report covers transactions only; master-data (GL) discrepancies are a stretch item.

---

7. Tier 3: Stretch (optional)

Part L

Pick what you can do **well**: GL master-data reconciliation (e.g. a Fineract GL with no ERP account), fees/penalties in the mapping, demonstrating overpayment/reversal, a dead-letter view, Prometheus metrics, scheduled reconciliation, signed partner webhooks.

---

8. Testing

Level	Minimum
Java unit + integration	the Part A customization (JUnit)
Python unit	validation, transaction mapping, accounting mapping, account-type mapping, idempotency logic, state transitions
Integration	Integration Service ↔ customized Fineract; Integration Service ↔ Mock ERP
Failure	Fineract unavailable, ERP unavailable, timeouts, duplicate request
Concurrency	parallel same-loan disbursements and repayments (I2, I3), written in Python

Python components use type hints checked by a type checker and a linter. One documented command per language runs its tests.

9. Observability

Structured JSON logs with: `correlation_id`, `external_reference`, `client_uid`, `loan_reference`, `fineract_transaction_id`, `erp_transaction_id`, `status`, `timestamp`.

In `DECISIONS.md`, show how an engineer answers **"What happened to FIN-2026-000001?"** from logs and one API call, with a sample log trail.

10. Documentation

```
README.md          start (docker compose up), tests per language, pinned versions, tiers
completed
ARCHITECTURE.md    components, data flow, data model, state machine, GL sync design
API.md             endpoints, examples, errors, idempotency contract
SIMULATIONS.md     every simulation, using the evidence template
DECISIONS.md       trade-offs and rationale
```

`DECISIONS.md` must cover: why the chosen Fineract customization mechanism, sync vs. async, database constraints, idempotency, retries, partial failure, what you chose not to build, and known weaknesses.

11. Evidence template (per simulation)

1. Initial state 2. Request 3. Response 4. Fineract state 5. ERP state 6. Integration state 7. Expected result 8. Actual result 9. Explanation

Prefer scripted, re-runnable simulations (e.g. `make simulate-duplicates`) that print before/after state. Screenshots and a Postman collection are supporting evidence only. Use **incremental commits**; a single giant commit is a review concern.

12. Repository layout (suggested)

```
fintech-fineract-assessment/
├── README.md  ARCHITECTURE.md  API.md  SIMULATIONS.md  DECISIONS.md
├── fineract-customization/   (patch/diff, Java tests, build notes, Dockerfile)
├── integration-service/      (app/, tests/, Dockerfile)
├── mock-erp/                 (app/, tests/, Dockerfile)
├── fineract/configuration/
├── scripts/
├── docker-compose.yml
├── postman/collection.json
└── evidence/
```

13. Evaluation

Area	Weight	Strong looks like
Fineract understanding	10%	Correct product/GL setup, uses `externalId`, reads real transaction breakdowns
Fineract customization (Java/Spring Boot)	15%	Real change inside Fineract, required behaviors enforced there, Liquibase, JUnit tests, builds from clean
Loan lifecycle	10%	Real state captured at every stage, ends Closed
Integration/API design (Python)	10%	Clear contract, no leaked Fineract IDs, errors, correlation IDs, typed code
Idempotency and concurrency	20%	DB-enforced, correct per-operation scope, tested in parallel
Failure handling	10%	Explicit state machine, safe retries, timeout resolved by lookup

Area	Weight	Strong looks like
ERP/accounting	10%	Balanced entries derived from Fineract data, synced GL accounts used
Reconciliation	5%	All four outcomes, clear keying
Testing and evidence	5%	Reproducible, automated, both languages
Code quality and docs	5%	Readable, honest about limits
Total	**100%**	

Red flags: retrying a financial call without first checking whether it happened; idempotency held only in application memory; journal entries computed independently of Fineract; exposing Fineract internal IDs as the public contract; a "Java customization" that is only a thin wrapper or dead code; claims in docs the evidence does not show.

14. Ground rules

- AI assistants and online references are allowed. **Disclose** where you used them; you must be able to explain and modify any part live.
- Do not use real personal or financial data.
- Ask about ambiguities. If unanswered, record your assumption in `DECISIONS.md` and proceed.

15. Follow-up interview (60 min)

- Walk through one failure scenario and your concurrency mechanism.
- **Live Java change:** e.g. add a field to the GL entity with its migration, or explain where in Fineract's command pipeline your logic runs and why your transaction boundary is correct.
- **Live Python change:** e.g. add a new account type and mapping, or a new failure case to the state machine, with tests.
- Extension question: "the ERP now returns 429s" or "add write-off."
- You may be asked to make customizations using AI